

# Lecture 1: The story of MiniMat

Imagine that a general-purpose CPU is a brilliant engineer sitting alone at a desk.

You give the engineer two matrices and ask for their product. They can absolutely calculate it - but they must repeatedly:

- 1 Fetch two numbers.
- 2 Multiply them.
- 3 Fetch another pair.
- 4 Multiply again.
- 5 Add the products.
- 6 Repeat for every output.
- 7 Store the result.

The engineer is flexible. Tomorrow, they can run an operating system, parse a file, control a robot, or play a game. But that flexibility costs time and energy.

Now imagine replacing that engineer, for one particular task, with a tiny factory. The factory contains several workers, each permanently equipped with a multiplier and an accumulator. Numbers move through the factory in a carefully timed rhythm. Multiple products are calculated simultaneously.

That factory is MiniMat.

---

## 1. What is a hardware accelerator?

A hardware accelerator is a circuit designed to perform a particular class of computation more efficiently than a general-purpose CPU.

Examples include:

- A GPU accelerating graphics and parallel arithmetic
- A video decoder accelerating H.264 or AV1 decoding
- An AES engine accelerating encryption
- A neural-processing unit accelerating matrix operations
- A digital signal processor accelerating filtering and transforms

A CPU is designed to do almost anything reasonably well. An accelerator is designed to do one category of work extremely well.

This specialization allows an accelerator to use:

- More parallelism
- Fewer instruction-fetch and decode operations
- Shorter, more regular data paths
- Local data reuse
- Precisely sized arithmetic
- Less unnecessary control logic

The downside is that the accelerator cannot do arbitrary work. MiniMat cannot run Linux, browse the internet, or compile C. It calculates matrix products.

That narrowness is not a weakness. It is the entire point.

## Is MiniMat a processor?

MiniMat is not a general-purpose processor like ARM or RISC-V. It does not fetch a sequence of instructions from memory.

It is better described as:

**A domain-specific matrix processor controlled through SPI.**

It processes data, contains pipelined computation, has state and control logic, and executes commands. But its "program" is essentially:

- Load matrix A
- Load matrix B
- Start
- Read matrix C

Later, you could place MiniMat beside a RISC-V core. The RISC-V processor would run software, while MiniMat would perform matrix calculations on its behalf.

## 2. Why matrix multiplication?

Matrix multiplication is important because many AI operations eventually become collections of dot products.

Suppose one neuron receives four inputs:

$$x_0, x_1, x_2, x_3$$

The neuron has four corresponding weights:

$$w_0, w_1, w_2, w_3$$

Before applying an activation function, the neuron calculates:

$$y = x_0w_0 + x_1w_1 + x_2w_2 + x_3w_3$$

That is a dot product.

A neural-network layer contains many neurons, each calculating a dot product. Matrix multiplication organizes all those dot products into one operation:

$$Y=XW$$

Matrix multiplication appears in:

- Fully connected neural-network layers
- Convolutional layers after rearranging image windows
- Transformer attention
- Image processing
- Digital filters
- Scientific computing
- Control and estimation algorithms

MiniMat implements the common mathematical engine beneath these applications. It does not implement a complete neural network, but it implements one of its most important computational building blocks.

### 3. What MiniMat calculates

MiniMat's physical array is 2 x 2, but this does not mean it can only multiply two ordinary 2 x 2 matrices.

It calculates:

$$A_{[2 \times K]} \times B_{[K \times 2]} = C_{[2 \times 2]}$$

For the first version, K can be between 1 and 8.

Consider:

$$A = \begin{bmatrix} 1 & -2 & 3 \\ 4 & 0 & -1 \end{bmatrix}$$

and:

$$B = \begin{bmatrix} 2 & 1 \\ -1 & 3 \\ 2 & -2 \end{bmatrix}$$

Here, K=3.

The output is:

$$C = \begin{bmatrix} C_{[00]} & C_{[01]} \\ C_{[10]} & C_{[11]} \end{bmatrix}$$

Each output is a dot product.

For example:

$$C_{[00]} = (1 \times 2) + (-2 \times -1) + (3 \times 2) = 10$$

$$C_{[01]} = (1 \times 1) + (-2 \times 3) + (3 \times -2) = -11$$

$$C_{[10]} = (4 \times 2) + (0 \times -1) + (-1 \times 2) = 6$$

$$C_{[11]} = (4 \times 1) + (0 \times 3) + (-1 \times -2) = 6$$

Therefore:

$$C = \begin{bmatrix} 10 & -11 \\ 6 & 6 \\ 1 \end{bmatrix}$$

MiniMat calculates all four dot products using four processing elements operating in parallel.

For a given K, the complete operation requires 4K multiply-accumulate operations. When K=8, MiniMat performs 32 MAC operations.

## 4. What is a MAC?

MAC means multiply-accumulate.

The operation is:

$$ACC \leftarrow ACC + A \times B$$

A MAC unit contains:

- A multiplier
- An adder
- An accumulator register

For example, suppose a MAC receives these operand pairs:

| Cycle | A  | B | Product | Accumulator |
|-------|----|---|---------|-------------|
| Start | -  | - | -       | 0           |
| 1     | 2  | 3 | 6       | 6           |
| 2     | -1 | 4 | -4      | 2           |
| 3     | 5  | 2 | 10      | 12          |

After three valid products, the accumulator contains 12.

This is exactly how one matrix output is calculated.

MiniMat has four processing elements, and every PE owns one MAC accumulator:

```
PE(0,0) calculates C(0,0)
PE(0,1) calculates C(0,1)
PE(1,0) calculates C(1,0)
PE(1,1) calculates C(1,1)
```

So the four workers in our factory each build one output result.

## 5. Why INT8 inputs?

MiniMat uses signed 8-bit integers.

An INT8 value ranges from:

-128 to 127

Neural networks are often trained using floating-point arithmetic, but inference can frequently use quantized integers.

Instead of representing a weight as something like:

0.734921

a quantized model may represent it with an integer such as:

94

along with a scale factor that explains what the integer means in the original numerical range.

INT8 arithmetic is attractive because:

- INT8 multipliers are smaller than floating-point multipliers.
- Each value requires only one byte of storage.
- More values can move through the same interconnect.
- More MAC units can fit in a given chip area.
- Power consumption can be lower.

### Why is the product 16 bits?

The largest positive product is:

$$(-128) \times (-128) = 16384$$

A signed 8-bit result cannot represent that. Therefore, the product needs approximately 16 signed bits.

### Why is the accumulator 32 bits?

A dot product adds several 16-bit products. Even when one product fits in 16 bits, their sum may not.

MiniMat therefore uses:

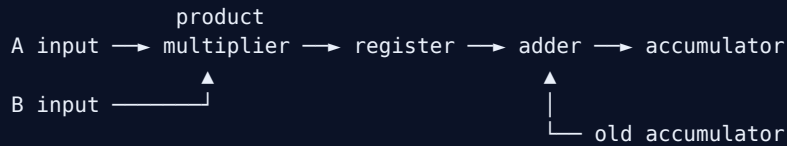
- 8-bit signed A
- 8-bit signed B
- 16-bit signed product
- 32-bit signed accumulator

For  $K \leq 8$ , 32 bits are considerably more than necessary, but that is useful for version one: it avoids complicated overflow concerns and returns the complete mathematical result.

## 6. Meet the processing element

A processing element, or PE, is one worker inside the MiniMat factory.

Conceptually, it contains:



It also forwards operands:

- A moves to the neighboring PE on the right.
- B moves to the neighboring PE below.

The PE therefore performs three jobs:

- 1 Multiply its current A and B values.
- 2 Add the product into its local accumulator.
- 3 Pass A and B to neighboring PEs.

The PE must also carry a valid signal.

The valid bit means:

**"The product currently in this pipeline stage represents real matrix data."**

This matters because some cycles contain bubbles while the array fills and drains. A zero value is not the same as invalid data: zero is a perfectly valid operand.

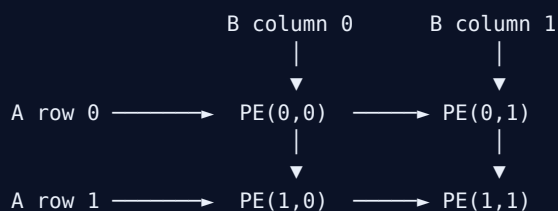
Never try to infer validity from whether the data equals zero.

## 7. The systolic array

"Systolic" comes from the rhythmic pumping action of a biological heart.

In a systolic array, data moves through a regular collection of processing elements one clock pulse at a time.

MiniMat's array is:



A values travel horizontally.

B values travel vertically.

Each PE keeps its output accumulator locally.

This is called output-stationary dataflow.

## Why "output stationary"?

During the calculation:

- A values move.
- B values move.
- Partial C values stay inside their owning PE.

For example,  $C_{[01]}$  remains inside  $PE(0,1)$  throughout the operation.

$PE(0,1)$  repeatedly performs:

$$C_{[01]} \leftarrow C_{[01]} + A_{[0k]}B_{[k1]}$$

Only after all  $K$  products have been processed is the completed result copied out.

Keeping the partial sum stationary reduces data movement. This matters because, in modern chips, moving data can consume as much or more energy than performing arithmetic.

## 8. Why the operands must be skewed

Suppose all  $A$  rows and  $B$  columns entered simultaneously without delays.

$PE(0,0)$  would receive matching operands immediately. But  $PE(0,1)$  receives  $A$  only after  $A$  has moved through  $PE(0,0)$ . Similarly,  $PE(1,0)$  receives  $B$  only after  $B$  has moved through  $PE(0,0)$ .

If everything enters at the same time, the off-diagonal PEs may combine operands from different  $k$  indices.

For correct multiplication,  $PE(i,j)$  must receive:

$$A[i][k] \text{ and } B[k][j]$$

on the same cycle.

Therefore:

- A row 0 begins immediately.
- A row 1 is delayed by one cycle.
- B column 0 begins immediately.
- B column 1 is delayed by one cycle.

This creates a diagonal wavefront.

For a conceptual  $K=3$  operation:

| Cycle | $PE(0,0)$ | $PE(0,1)$ | $PE(1,0)$ | $PE(1,1)$ |
|-------|-----------|-----------|-----------|-----------|
| 0     | $k_0$     | -         | -         | -         |
| 1     | $k_1$     | $k_0$     | $k_0$     | -         |
| 2     | $k_2$     | $k_1$     | $k_1$     | $k_0$     |
| 3     | -         | $k_2$     | $k_2$     | $k_1$     |
| 4     | -         | -         | -         | $k_2$     |

At cycle 2, all four PEs are performing useful work. This is the moment when the array reaches its peak throughput of four MAC operations per cycle.

This table describes the spatial wavefront. If the PE includes an extra product pipeline register, the accumulator updates occur later - but the operand pairings remain the same.

That distinction will become important:

- The array schedule determines which operands meet.
- The PE pipeline determines when their product reaches the accumulator.

## 9. What pipelining means here

Imagine a worker who must perform two activities:

- 1 Multiply A and B.
- 2 Add the product to the accumulator.

An unpipelined PE performs both operations in one clock period:

```
register -> multiplier -> 32-bit adder -> accumulator register
```

The clock must be slow enough for both the multiplier and adder to finish.

A pipelined PE separates the work:

```
Stage 1: multiply A and B, then register the product
Stage 2: add the registered product into the accumulator
```

Now the clock period only needs to accommodate approximately one major arithmetic stage at a time.

The result is usually:

- Higher possible clock frequency
- More registers
- Higher clock-tree load
- Slightly more area
- Additional latency
- More complicated valid alignment

### Latency versus throughput

These terms are frequently confused.

Latency answers:

**How long does one complete matrix operation take?**

Throughput answers:

**Once the pipeline is active, how much useful arithmetic happens per cycle?**

Pipelining may increase latency slightly because data passes through an additional register. But it can improve throughput because the clock can run faster and new products can enter on consecutive cycles.

A useful analogy is laundry:

- Washing and drying one load sequentially may take two hours.

- With separate washer and dryer stages, the first load still takes two hours.
- But after the pipeline fills, a new load can finish every hour.

MiniMat works similarly.

## 10. The complete journey of one matrix operation

Let us follow one job from beginning to end.

### Scene 1: MiniMat waits

The accelerator is idle.

Its status might say:

```
BUSY = 0  
DONE = 0  
ERROR = 0
```

The accumulators and buffers are either empty or contain old data that is not currently active.

### Scene 2: The host sends K

The host sends an SPI command saying:

"The upcoming dot products have length 3."

MiniMat checks that K is between 1 and 8 and stores it.

### Scene 3: The host loads A

The host sends six signed bytes because A is  $2 \times 3$ :

```
1, -2, 3, 4, 0, -1
```

MiniMat stores them in its A buffer.

### Scene 4: The host loads B

The host sends another six bytes:

```
2, 1, -1, 3, 2, -2
```

MiniMat stores them in its B buffer.

At this moment, SPI communication is finished. The accelerator does not need the slow serial interface during computation.

## Scene 5: The host sends START

The command controller checks:

- Is K legal?
- Has A been loaded?
- Has B been loaded?
- Is the accelerator currently idle?

If everything is valid, it accepts START.

## Scene 6: Clear

Before issuing operands, the controller clears all four PE accumulators.

This prevents results from a previous calculation from contaminating the next calculation.

## Scene 7: Issue the wavefront

The controller begins reading operands from the A and B buffers.

It adds the required row and column delays and marks valid operands. Values begin traveling through the 2 x 2 PE array.

## Scene 8: Drain the pipeline

After the controller has issued the final k value, it cannot immediately declare completion.

The final operands are still moving through the array and the product pipeline. The controller enters a drain state and waits for the final valid token to reach the last accumulation point.

## Scene 9: Capture results

The completed accumulators are copied into stable output registers:

```
C00 = 10  
C01 = -11  
C10 = 6  
C11 = 6
```

MiniMat lowers BUSY and raises sticky DONE.

Sticky means DONE remains asserted until software clears it or begins another defined operation. A one-cycle DONE pulse would be easy for a slow SPI host to miss.

## Scene 10: The host reads C

The host sends the READ\_C command.

MiniMat serializes the four 32-bit results onto MISO. The host reconstructs the numbers and compares them against its software model.

That is the complete life of one MiniMat job.

# 11. The important blocks

From a Verilog perspective, the design is a small collection of familiar structures.

## SPI slave

You already understand much of this block.

Its responsibility is to:

- Recognize SPI Mode 0 timing
- Receive command bytes
- Receive payload bytes
- Return status and result bytes
- Detect packet boundaries using chip select

For version one, the system clock can oversample the external SPI signals. This keeps the chip in one internal clock domain.

For example:

- System clock: 20 MHz
- SPI clock: 1 MHz

The faster system clock synchronizes SPI inputs and detects their transitions.

## Command controller

This is an FSM that interprets received bytes.

It decides whether a byte means:

- Set K
- Load A
- Load B
- Start
- Read status
- Read results
- Clear
- Read design identity

It also rejects malformed or illegal operations.

## Operand buffers

These hold the input matrices.

For  $K_{[MAX]}=8$ :

- A holds  $2 \times 8=16$  bytes.
- B holds  $8 \times 2=16$  bytes.

That is 32 bytes total.

The buffers allow SPI loading and matrix execution to happen at very different speeds.

## Systolic controller

This is the conductor of the orchestra.

It controls:

- Accumulator clear
- Buffer addresses
- Row and column delays
- Valid signals
- Issue count
- Pipeline drain
- Result capture
- BUSY and DONE

It performs very little arithmetic, but a mistake here can corrupt the entire calculation.

## Four processing elements

These perform the real computation.

Every PE contains:

- A and B forwarding registers
- An 8 x 8 signed multiplier
- A product pipeline register
- A 32-bit accumulator
- Valid-pipeline state

## Result registers

These capture the four completed C values.

They keep results stable while the slow SPI interface reads them byte by byte.

# 12. What MiniMat can do

MiniMat directly calculates four dot products of length K.

That already supports several interpretations.

## A tiny fully connected layer

A can represent two input samples, each containing K features.

B can represent weights for two output neurons.

The result contains:

- Output of neuron 0 for sample 0
- Output of neuron 1 for sample 0
- Output of neuron 0 for sample 1
- Output of neuron 1 for sample 1

MiniMat therefore computes a tiny batch of a neural-network layer.

## Larger matrix multiplication

Suppose you want to calculate a 4 x 4 output.

You divide that output into four 2 x 2 tiles:

```
C = [ C_tile_00 C_tile_01
      C_tile_10 C_tile_11 ]
```

The host runs MiniMat once for each output tile.

MiniMat remains physically 2 x 2, but software reuses it.

## K larger than 8

If the dot-product length is greater than eight, the host splits it into chunks.

For example, a length-16 dot product becomes:

```
C=C_[first 8]+C_[second 8]
```

Version one can return the two partial matrices to the host, which adds them.

A future version could allow the accelerator to preserve its accumulators across multiple chunks.

## Convolution

A convolution can be rearranged into dot products.

An image window becomes one vector, and a filter becomes another. MiniMat can compute those dot products, but the host must perform:

- Window extraction
- Padding
- Stride handling
- Channel organization

So MiniMat contains the convolution arithmetic, not the complete convolution engine.

## DSP operations

The same structure can perform:

- Correlation
- Small finite impulse response calculations
- Vector similarity
- Image transformations
- Batched dot products

It is an AI-relevant accelerator, but it is not limited exclusively to AI.

## 13. What MiniMat cannot do

It is important to understand the boundaries honestly.

Version one does not:

- Train a neural network
- Run a complete large model
- Store thousands of weights
- Perform floating-point arithmetic
- Automatically tile large matrices
- Apply bias or ReLU
- Transfer data using AXI or DMA
- Compete with a desktop GPU
- Hide the cost of loading data over SPI

In fact, for such small matrices, SPI communication will probably take much longer than the calculation.

That does not make the project unsuccessful.

The goal is not to beat a laptop. The goal is to demonstrate the architecture and build the complete silicon flow correctly.

A useful performance report should separate:

- Compute time inside the systolic array
- SPI loading time
- SPI result-reading time
- Complete end-to-end time

This shows that you understand the difference between peak arithmetic throughput and system performance.

## 14. Why verification is especially important

The multiplication itself is not the hardest part.

A multiplier producing the wrong answer usually fails obviously. A systolic scheduling bug can be more deceptive.

For example:

- PE(0,0) may be correct.
- PE(0,1) may combine A at k1 with B at k0.
- PE(1,0) may begin one cycle early.
- PE(1,1) may miss only the final product.
- The design may work for K=1 but fail for K=8.
- Positive numbers may work while negative numbers fail.
- RTL may work while reset behavior fails in silicon.

The most dangerous bugs involve identity through time:

**Which matrix element is this value, and which k index does it represent on this cycle?**

That is why valid signals and cycle tables matter.

Your verification journey should move outward:

- 1 Verify a single PE.
- 2 Verify four connected PEs.
- 3 Verify operand skewing.
- 4 Verify controller completion.
- 5 Verify buffers.
- 6 Verify SPI.
- 7 Verify the entire load-start-read sequence.
- 8 Verify the synthesized and post-layout design.

The reference model should calculate matrix multiplication in the obvious mathematical way. It should not copy the systolic algorithm, because then both models might share the same scheduling mistake.

## 15. Why the physical-design stage is meaningful

When writing RTL, the multiplier appears as one multiplication operation.

After synthesis, it becomes a network of gates.

The accumulator becomes:

- Adder cells
- Flip-flops
- Clock routing
- Reset routing

The four PEs become hundreds or thousands of physical objects connected by metal wires.

During physical design, you will discover that architecture is not merely abstract.

### Timing

The multiplier may create the longest combinational path.

Adding a product pipeline register may shorten that path, permitting a faster clock.

### Area

Four multipliers consume significantly more area than one. Operand buffers and 32-bit accumulators also require storage.

### Clocking

Every pipeline register must receive the clock. More pipelining means a larger clock tree and potentially more clock power.

### Routing

The placer and router must connect:

- A paths moving horizontally
- B paths moving vertically
- Control and valid signals
- Accumulators and result paths
- SPI and buffer logic

## Physical verification

Before the layout is acceptable, you need:

- Clean design-rule checking
- Layout matching the synthesized netlist
- Clean setup timing
- Clean hold timing
- Antenna compliance
- A valid final GDSII database

This is why MiniMat is a good physical-design project: its arithmetic and repetition create real physical tradeoffs without making the system impossibly large.

# 16. How MiniMat can scale

Once version one works, several directions become possible.

## Increase K

Increasing  $K_{[MAX]}$  allows longer dot products.

The PE count does not change, but you need:

- Larger operand buffers
- Wider addressing counters
- Longer computation time

This is primarily a storage scaling problem.

## Increase the array from 2 x 2 to 4 x 4

A 4 x 4 array contains 16 PEs.

Peak compute rises from:

- Four MACs per cycle
- To sixteen MACs per cycle

But the costs also rise:

- Approximately four times as many PEs
- More clock-tree load
- More routing congestion
- More input bandwidth
- More accumulators
- More verification combinations
- A longer fill and drain distance

This is not merely changing a parameter and declaring victory. The rest of the architecture must keep the larger array supplied with data.

## Double buffering

Version one works like this:

```
load -> compute -> read -> load next job
```

With double buffers:

```
compute buffer 0 while loading buffer 1
```

Communication and computation overlap, improving throughput.

But now you need ownership logic to ensure the host never writes the buffer currently being consumed.

## Add quantization and activation

A more complete neural-layer accelerator could perform:

- 1 Matrix multiplication
- 2 Add bias
- 3 Arithmetic right shift
- 4 Round
- 5 Saturate to INT8
- 6 Apply ReLU

This would produce data ready for the next neural layer.

It also introduces many numerical corner cases, particularly negative rounding and saturation.

## Add AXI

An AXI4-Lite wrapper would allow MiniMat to become a peripheral inside a larger FPGA or SoC.

SPI would no longer be the only access method. A CPU could write control registers and buffers through a memory-mapped bus.

## Add streaming and DMA

For larger arrays, manually writing every operand through control registers becomes inefficient.

An AXI-Stream or DMA interface could move large blocks of activations and weights from memory.

At that point, memory bandwidth may become more important than multiplication.

## Add a scalar processor

Eventually, you could place a small RISC-V or custom controller beside MiniMat.

The processor would:

- Interpret a tiny program
- Configure matrix jobs
- Select buffers
- Apply activation functions
- Schedule repeated tiles
- Handle communication

Then MiniMat becomes the compute engine inside a small AI SoC.

That is a good future project - but it should come after the standalone accelerator is complete.

# 17. The five ideas you should truly understand

Before touching RTL, make sure these ideas feel natural.

## 1. Dot product

One result equals a sum of pairwise products.

## 2. MAC

A hardware unit repeatedly performs:

$$\text{ACC} \leftarrow \text{ACC} + A \times B$$

## 3. Systolic movement

Operands advance between neighboring PEs on clock cycles.

## 4. Output-stationary dataflow

Each partial C result stays in one PE while A and B move.

## 5. Valid alignment

Every data value has an identity and validity that must travel through the same pipeline latency.

If you understand those five points, the project stops looking like mysterious AI hardware. It becomes a collection of registers, arithmetic, muxes, counters, and FSMs - the concepts you already know from Verilog and computer architecture.

# 18. What to study before implementation

I recommend this learning order:

- 1 Perform several  $2 \times K$  multiplied by  $K \times 2$  calculations by hand.
- 2 Rewrite each output as a dot product.
- 3 Draw the four PEs and assign one C output to each.
- 4 Draw where every A and B value moves each cycle.
- 5 Create a cycle table for  $K=1$ ,  $K=2$ , and  $K=3$ .
- 6 Add one product pipeline stage and redraw when accumulation occurs.
- 7 Study signed two's-complement multiplication and expression-width rules.
- 8 Define the controller states and the meaning of BUSY and DONE.
- 9 Write the SPI command behavior in ordinary English.
- 10 Only then begin designing RTL.

The most important artifact before RTL is a correct cycle diagram.

If the architecture cannot be explained cycle by cycle on paper, implementing it in Verilog will only hide the confusion behind waveforms.

# The final mental picture

MiniMat is a four-worker mathematical factory.

SPI delivers the raw material: two small matrices.

The operand buffers store it.

The controller starts the rhythm.

A values march from left to right.

B values march from top to bottom.

Each PE multiplies the pair that meets there and adds the product to its private accumulator.

A diagonal wave of useful work passes through the array.

After the final wave drains, four completed dot products are captured.

SPI carries them back to the host.

Then the entire architecture - multipliers, registers, controller, buffers and protocol logic - is synthesized into gates, placed as physical cells, connected with metal wires, checked for timing and manufacturing rules, and exported as GDSII.

That is MiniMat: small enough to understand completely, but deep enough to teach you how a real AI accelerator moves from an equation to silicon.